

```

import flet as ft
import os
import json
import threading
import asyncio
import hashlib
import datetime
import webbrowser
from pathlib import Path
from dotenv import load_dotenv

load_dotenv(dotenv_path=Path(__file__).parent / ".env")

# Import the orchestrator class
from orchestrator import MalwareScannerOrchestrator
from llm_analyzer import analyze_sandbox_with_llm
import sys
from report_generator import build_html_report, compute_hashes

if getattr(sys, 'frozen', False):
    base_path = Path(sys.executable).parent # looks next to .exe
else:
    base_path = Path(__file__).parent

# — Colors —————
BG_COLOR    = "#121212"
CARD_BG     = "#1E1E1E"
NEON_GREEN  = "#00FF00"
CYAN        = "#00FFFF"
NEON_RED    = "#FF0033"
YELLOW      = "#FFD700"
TEXT_COLOR  = "#FFFFFF"
PURPLE      = "#BF5FFF"

async def main(page: ft.Page):
    page.title = "Neurscan - Advanced Malware Orchestrator"
    page.theme_mode = ft.ThemeMode.DARK
    page.bgcolor = BG_COLOR
    page.padding = 20
    page.scroll = ft.ScrollMode.AUTO

    # — State —————
    selected_file = None
    last_result = None # stores the FinalVerdict after a scan

    # — Header —————
    title = ft.Text("Neurscan Orchestrator", size=32, weight=ft.FontWeight.BOLD, color=CYAN)

    file_path_text = ft.Text("No file selected", color=ft.Colors.GREY_400, italic=True)
    status_text = ft.Text("", color=CYAN, italic=True)
    progress_bar = ft.ProgressBar(width=400, visible=False, color=CYAN,
    bgcolor=ft.Colors.GREY_800)
    sandbox_checkbox = ft.Checkbox(label="Enable Falcon Sandbox Detonation (Slow)", value=False,
    fill_color=CYAN)
    quarantine_checkbox = ft.Checkbox(label="Auto-Quarantine High Risk", value=False,
    fill_color=CYAN)

    # — Verdict banner —————
    verdict_text = ft.Text(size=24, weight=ft.FontWeight.BOLD, color=ft.Colors.BLACK)
    verdict_banner = ft.Container(
        visible=False,
        padding=15,
        border_radius=10,

```

```

        alignment=ft.Alignment.CENTER,
        content=verdict_text,
    )

# — Scanner cards —————
def create_card(title_text):
    return ft.Card(
        bgcolor=CARD_BG,
        elevation=5,
        content=ft.Container(
            padding=15,
            width=250,
            content=ft.Column([
                ft.Text(title_text, size=18, weight=ft.FontWeight.BOLD, color=CYAN),
                ft.Text("Waiting for scan...")
            ])
        )
    )

yara_card = create_card("YARA")
ml_card = create_card("ML / EMBER")
vt_card = create_card("VirusTotal")
sandbox_card = create_card("Sandbox")

cards_row = ft.Row(
    [yara_card, ml_card, vt_card, sandbox_card],
    wrap=True,
    alignment=ft.MainAxisAlignment.START,
    visible=False
)

# — Tactics row —————
tactics_wrap = ft.Row(wrap=True, visible=False)
tactics_container = ft.Column([
    ft.Text("Behavioral Tactics Detected", size=20, weight=ft.FontWeight.BOLD, color=CYAN),
    tactics_wrap
], visible=False)

# — AI Analysis panel —————
ai_threat_name = ft.Text("", size=20, weight=ft.FontWeight.BOLD, color=NEON_RED)
ai_severity_chip = ft.Text("", size=14, weight=ft.FontWeight.BOLD)
ai_confidence = ft.Text("", size=12, color=ft.Colors.GREY_400)
ai_summary = ft.Text("", size=13, color=TEXT_COLOR, selectable=True)

ai_capabilities_col = ft.Column([], spacing=3)
ai_techniques_col = ft.Column([], spacing=3)
ai_iocs_col = ft.Column([], spacing=3)
ai_recommendation = ft.Text("", size=13, color=YELLOW, selectable=True, italic=True)

# Raw JSON toggle (collapsed by default)
raw_json_text = ft.Text(font_family="Consolas", size=11, selectable=True,
color=ft.Colors.GREY_400)
raw_json_tile = ft.ExpansionTile(
    title=ft.Text("Show Raw Sandbox JSON", size=13, color=ft.Colors.GREY_500),
    affinity=ft.TileAffinity.LEADING,
    collapsed_text_color=ft.Colors.GREY_600,
    text_color=ft.Colors.GREY_400,
    controls=[
        ft.Container(
            content=raw_json_text,
            bgcolor=ft.Colors.BLACK,
            padding=10,
            border_radius=5,
            border=ft.Border.all(1, ft.Colors.GREY_800),
        )
    ]
)

```

```

    ],
)

ai_panel = ft.Container(
    visible=False,
    bgcolor=CARD_BG,
    border_radius=10,
    border=ft.Border.all(1, PURPLE),
    padding=20,
    content=ft.Column([
        ft.Row([
            ft.Icon(ft.Icons.PSYCHOLOGY, color=PURPLE, size=24),
            ft.Text("AI Threat Intelligence", size=20, weight=ft.FontWeight.BOLD, color=PURPLE),
        ]),
        ft.Divider(color=PURPLE, height=1),
        ft.Row([
            ai_threat_name,
            ft.Container(width=12),
            ai_severity_chip,
            ft.Container(width=12),
            ai_confidence,
        ], wrap=True),
        ft.Container(height=8),
        ft.Text("Summary", size=13, weight=ft.FontWeight.BOLD, color=CYAN),
        ai_summary,
        ft.Container(height=10),
        ft.Row([
            ft.Column([
                ft.Text("Capabilities", size=13, weight=ft.FontWeight.BOLD, color=CYAN),
                ai_capabilities_col,
            ], expand=True),
            ft.Column([
                ft.Text("ATT&CK Techniques", size=13, weight=ft.FontWeight.BOLD, color=CYAN),
                ai_techniques_col,
            ], expand=True),
            ft.Column([
                ft.Text("IOCs", size=13, weight=ft.FontWeight.BOLD, color=CYAN),
                ai_iocs_col,
            ], expand=True),
        ], spacing=20),
        ft.Container(height=10),
        ft.Row([
            ft.Icon(ft.Icons.LIGHTBULB, color=YELLOW, size=16),
            ft.Text("Recommendation", size=13, weight=ft.FontWeight.BOLD, color=YELLOW),
        ]),
        ai_recommendation,
        ft.Container(height=10),
        raw_json_tile,
    ], spacing=6),
)

```

— Helpers

```

def _severity_color(sev: str) -> str:
    return {
        "CRITICAL": NEON_RED,
        "HIGH":     "#FF6600",
        "MEDIUM":  YELLOW,
        "LOW":      CYAN,
        "CLEAN":    NEON_GREEN,
    }.get(sev.upper(), ft.Colors.GREY_400)

def _bullet(text: str, color=TEXT_COLOR) -> ft.Text:
    return ft.Text(f"• {text}", size=12, color=color, selectable=True)

def populate_ai_panel(analysis: dict):

```

```

"""Fill the AI panel controls with data from the LLM response."""
sev = analysis.get("severity", "UNKNOWN")

ai_threat_name.value = analysis.get("threat_name", "Unknown")
ai_threat_name.color = _severity_color(sev)
ai_severity_chip.value = f"[{sev}]"
ai_severity_chip.color = _severity_color(sev)
ai_confidence.value = f"Confidence: {analysis.get('confidence', '?')}"
ai_summary.value = analysis.get("summary", "")
ai_recommendation.value = analysis.get("recommendation", "")

ai_capabilities_col.controls.clear()
for cap in analysis.get("capabilities", []) or ["-"]:
    ai_capabilities_col.controls.append(_bullet(cap))

ai_techniques_col.controls.clear()
for tech in analysis.get("attack_techniques", []) or ["-"]:
    ai_techniques_col.controls.append(_bullet(tech, YELLOW))

ai_iocs_col.controls.clear()
for ioc in analysis.get("iocs", []) or ["-"]:
    ai_iocs_col.controls.append(_bullet(ioc, NEON_RED))

ai_panel.visible = True

# — File picker —————
# In Flet 0.85, FilePicker.pick_files() is async and returns a list directly.
file_picker = ft.FilePicker()
page.services.append(file_picker)

def update_status(msg):
    status_text.value = msg
    page.update()

is_scanning = False

async def stop_scan(e):
    nonlocal is_scanning
    if not is_scanning: return
    is_scanning = False
    update_status("Scan stopped by user.")
    status_text.color = YELLOW
    progress_bar.visible = False
    scan_button.visible = True
    stop_button.visible = False
    page.update()

# — Select file button handler (async) —————
async def pick_file(e):
    nonlocal selected_file
    files = await file_picker.pick_files(allow_multiple=False)
    if files and len(files) > 0:
        selected_file = files[0].path
        file_path_text.value = f"Selected: {selected_file}"
        file_path_text.color = CYAN
    else:
        selected_file = None
        file_path_text.value = "No file selected"
        file_path_text.color = ft.Colors.GREY_400
    page.update()

# — Scan logic —————
def start_scan(e):
    nonlocal selected_file, is_scanning
    if not selected_file:

```

```

        status_text.value = "Please select a file first!"
        status_text.color = NEON_RED
        page.update()
        return

    is_scanning = True

    # Reset UI
    status_text.color = CYAN
    progress_bar.visible = True
    cards_row.visible = False
    tactics_container.visible = False
    ai_panel.visible = False
    verdict_banner.visible = False
    report_button.visible = False
    tactics_wrap.controls.clear()
    ai_capabilities_col.controls.clear()
    ai_techniques_col.controls.clear()
    ai_iocs_col.controls.clear()

    for card in [yara_card, ml_card, vt_card]:
        card.content.content.controls[1].value = "Scanning..."
        card.content.content.controls[1].color = TEXT_COLOR
    sandbox_card.content.content.controls[1].value = "Scanning..." if sandbox_checkbox.value else
"Skipped"
    sandbox_card.content.content.controls[1].color = TEXT_COLOR if sandbox_checkbox.value else
ft.Colors.GREY_400

    scan_button.visible = False
    stop_button.visible = True
    page.update()

    # Run the blocking orchestrator in a thread pool to keep UI responsive
    loop = asyncio.get_event_loop()
    await loop.run_in_executor(None, lambda: _run_orchestrator())

def _run_orchestrator():
    """Blocking scan logic – runs in a background thread."""
    nonlocal is_scanning
    try:
        vt_key = os.getenv("VT_API_KEY")
        falcon_key = os.getenv("HYBRID_ANALYSIS_API_KEY")

        if getattr(sys, 'frozen', False):
            _base = Path(sys._MEIPASS)
        else:
            _base = Path(__file__).parent

        model_env = os.getenv("MODEL_PATH", "ember_model_2018.txt")
        model_path = str(_base / Path(model_env).name)
        rules_dir_path = str(_base / "rules")

        update_status("Initializing Orchestrator...")
        if not is_scanning: return
        orchestrator = MalwareScannerOrchestrator(
            vt_api_key=vt_key,
            falcon_api_key=falcon_key,
            model_path=model_path,
            rules_dir=rules_dir_path
        )

        result = orchestrator.scan_file(
            filepath=selected_file,
            auto_quarantine=quarantine_checkbox.value,
            quarantine_threshold=75.0,

```

```

        run_sandbox=sandbox_checkbox.value,
        progress_callback=update_status
    )

    if not is_scanning: return

    # — Update scanner cards —————
    cards_row.visible = True

    # YARA
    if result.yara_result:
        yr = result.yara_result
        color = NEON_GREEN if yr.verdict == "CLEAN" else NEON_RED
        yara_card.content.content.controls[1].value = f"Score: {yr.total_score}/100\nVerdict:
{yr.verdict}"
        yara_card.content.content.controls[1].color = color
    else:
        yara_card.content.content.controls[1].value = "N/A"

    # ML
    if result.ml_score is not None:
        ml_display = result.ml_score * 100
        color = NEON_RED if ml_display > 50 else NEON_GREEN
        label = "Malicious" if ml_display > 50 else "Benign"
        ml_card.content.content.controls[1].value = f"Score: {ml_display:.1f}/100\n{label}"
        ml_card.content.content.controls[1].color = color
    elif result.ml_error:
        ml_card.content.content.controls[1].value = f"Error: {result.ml_error}"
        ml_card.content.content.controls[1].color = NEON_RED
    elif not result.is_pe:
        ml_card.content.content.controls[1].value = "Skipped (Not a PE file)"
        ml_card.content.content.controls[1].color = ft.Colors.GREY_400
    else:
        ml_card.content.content.controls[1].value = "Skipped (Model file missing)\nPlace
ember_model_2018.txt in malware_tools/"
        ml_card.content.content.controls[1].color = YELLOW

    # VT
    if result.vt_result:
        vt = result.vt_result
        color = NEON_RED if vt.detections > 0 else NEON_GREEN
        vt_card.content.content.controls[1].value = f"{vt.detections}/{vt.total_engines}
({vt.detection_rate}%)\n{vt.threat_label or 'Clean'}"
        vt_card.content.content.controls[1].color = color
    elif not vt_key:
        vt_card.content.content.controls[1].value = "Skipped (No API Key)\nAdd VT_API_KEY to
.env"
        vt_card.content.content.controls[1].color = YELLOW
    else:
        vt_card.content.content.controls[1].value = "Skipped"
        vt_card.content.content.controls[1].color = ft.Colors.GREY_400

    # Sandbox card + tactics + AI analysis
    if result.sandbox_result:
        exec_summary = result.sandbox_result.get("Executive_Summary", {})
        s_score = exec_summary.get("Threat_Score", 0)
        s_verdict = exec_summary.get("Verdict", "Unknown").upper()
        color = NEON_RED if s_score >= 50 else NEON_GREEN
        sandbox_card.content.content.controls[1].value = f"Score: {s_score}/100\n{s_verdict}"
        sandbox_card.content.content.controls[1].color = color

        behaviors = result.sandbox_result.get("Behavioral_Analysis", {})
        if behaviors:
            tactics_container.visible = True
            for tactic in behaviors.keys():

```

```

        tactics_wrap.controls.append(
            ft.Chip(
                label=ft.Text(tactic, color=ft.Colors.BLACK,
weight=ft.FontWeight.BOLD),
                bgcolor=CYAN
            )
        )

        raw_json_text.value = json.dumps(result.sandbox_result, indent=2)

        update_status("Running AI threat analysis...")
        groq_key = os.getenv("GROQ_API_KEY")
        analysis = analyze_sandbox_with_llm(result.sandbox_result, api_key=groq_key)
        if analysis:
            populate_ai_panel(analysis)

    elif sandbox_checkbox.value:
        if not falcon_key:
            sandbox_card.content.content.controls[1].value = "Skipped (No API Key)\nAdd
HYBRID_ANALYSIS_API_KEY to .env"
            sandbox_card.content.content.controls[1].color = YELLOW
        else:
            sandbox_card.content.content.controls[1].value = "Error / No Data"
            sandbox_card.content.content.controls[1].color = NEON_RED

# — Final verdict banner —————
verdict_banner.visible = True
if result.combined_score >= 50:
    verdict_banner.bgcolor = NEON_RED
    verdict_text.value = f"MALICIOUS (Score: {result.combined_score:.1f}/100) -
{result.confidence_level}"
else:
    verdict_banner.bgcolor = NEON_GREEN
    verdict_text.value = f"CLEAN (Score: {result.combined_score:.1f}/100) -
{result.confidence_level}"

if result.quarantined:
    verdict_text.value += " [QUARANTINED]"

# Store result for report generation and show button
nonlocal last_result
last_result = result
report_button.visible = True

update_status("Scan Complete!")

except Exception as ex:
    if is_scanning:
        update_status(f"Error during scan: {str(ex)}")
        status_text.color = NEON_RED
finally:
    if is_scanning:
        is_scanning = False
        progress_bar.visible = False
        scan_button.visible = True
        stop_button.visible = False
        page.update()

# — Generate Report handler —————
async def generate_report_click(e):
    if not last_result or not selected_file:
        update_status("No scan results to generate a report for.")
        return

    update_status("Generating HTML report...")

```

```

report_button.disabled = True
page.update()

try:
    scan_time = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    hashes = compute_hashes(selected_file)
    html = build_html_report(last_result, selected_file, hashes, scan_time)

    out_path = Path(selected_file).parent /
f"Neurscan_Report_{Path(selected_file).stem}.html"
    out_path.write_text(html, encoding="utf-8")

    update_status(f"Report saved: {out_path}")
    webbrowser.open(str(out_path.resolve()))

except Exception as ex:
    update_status(f"Report generation failed: {ex}")
    status_text.color = NEON_RED
finally:
    report_button.disabled = False
    page.update()

# — Buttons —————
scan_button = ft.ElevatedButton(
    "Start Scan",
    icon=ft.Icons.PLAY_ARROW,
    color=BG_COLOR,
    bgcolor=CYAN,
    on_click=start_scan,
    style=ft.ButtonStyle(
        shape=ft.RoundedRectangleBorder(radius=5),
        animation_duration=300
    )
)

stop_button = ft.ElevatedButton(
    "Stop Scan",
    icon=ft.Icons.STOP,
    color=BG_COLOR,
    bgcolor=NEON_RED,
    visible=False,
    on_click=stop_scan,
    style=ft.ButtonStyle(
        shape=ft.RoundedRectangleBorder(radius=5),
        animation_duration=300
    )
)

report_button = ft.ElevatedButton(
    "Generate Report",
    icon=ft.Icons.DESCRPTION,
    color=BG_COLOR,
    bgcolor=NEON_GREEN,
    visible=False,
    on_click=generate_report_click,
    style=ft.ButtonStyle(
        shape=ft.RoundedRectangleBorder(radius=5),
        animation_duration=300
    )
)

select_btn = ft.ElevatedButton(
    "Select Sample",
    icon=ft.Icons.FOLDER_OPEN,
    color=TEXT_COLOR,

```



```

        bgcolor=ft.Colors.BLUE_GREY_800,
        on_click=pick_file,
        style=ft.ButtonStyle(
            shape=ft.RoundedRectangleBorder(radius=5),
            animation_duration=300
        )
    )

# — Layout —————
page.add(
    title,
    ft.Divider(color=CYAN),
    ft.Row([select_btn, file_path_text], alignment=ft.MainAxisAlignment.START),
    ft.Row([sandbox_checkbox, quarantine_checkbox]),
    ft.Container(height=10),
    ft.Row([scan_button, stop_button]),
    ft.Container(height=10),
    status_text,
    progress_bar,
    ft.Container(height=10),
    verdict_banner,
    report_button,
    ft.Container(height=10),
    cards_row,
    ft.Container(height=10),
    tactics_container,
    ft.Container(height=10),
    ai_panel,
)

if __name__ == "__main__":
    ft.run(main)

```